



Práctica 1

Materia:
Seguridad en el Desarrollo de Aplicaciones

Profesor
Gerardo Ramírez Villarreal

Grupo:
IDGS17

Alumn@:
Sánchez Palomino Nuvia Yasmin

Matricula:
2023371105

Índice

Introducción	3
Tabla comparativa OWASP Top 10	4
Casos reales	5
Caso 1 — Equifax (2017)	5
Caso 2 — SolarWinds (2020)	5
Código vulnerable vs seguro	6
1. SQL Injection	6
2. Broken Access Control	6
3. Contraseñas inseguras	7
Ranking personal	8
Conclusión	9
Referencias	10

Introducción

El OWASP Top 10 representa una de las referencias más importantes en el ámbito de la ciberseguridad, ya que identifica las diez vulnerabilidades más críticas y comunes presentes en aplicaciones web. Este listado es utilizado por desarrolladores, empresas y especialistas en seguridad para comprender los principales riesgos existentes, así como las medidas necesarias para prevenirlos y mitigarlos.

La práctica tiene como objetivo investigar y analizar las vulnerabilidades incluidas en el OWASP Top 10 2023, comprendiendo su funcionamiento, impacto y posibles consecuencias dentro de un sistema vulnerable. Además, se revisarán casos reales de ataques informáticos relacionados con estas vulnerabilidades y se presentarán ejemplos de código vulnerable y seguro, con la finalidad de fortalecer el conocimiento sobre desarrollo seguro y concientizar acerca de la importancia de implementar mecanismos de protección adecuados en las aplicaciones modernas.

Tabla comparativa OWASP Top 10

Número	Vulnerabilidad	Descripción	Ejemplo de ataque	Impacto	Mitigación
A01	Broken Access Control	Usuarios acceden a recursos sin permisos	Ver cuentas de otros usuarios cambiando IDs	Robo de datos	Validar permisos backend
A02	Cryptographic Failures	Mala protección de datos sensibles	Contraseñas sin cifrar	Fuga de información	HTTPS + hash seguro
A03	Injection	Entrada maliciosa ejecuta comandos	SQL Injection	Robo/modificación BD	Queries parametrizadas
A04	Insecure Design	Diseño inseguro desde arquitectura	Reset password inseguro	Toma de cuentas	Threat modeling
A05	Security Misconfiguration	Configuración insegura	Panel admin público	Acceso no autorizado	Hardening
A06	Vulnerable Components	Librerías vulnerables	Uso de Log4j vulnerable	RCE	Actualizaciones
A07	Identification Failures	Problemas de autenticación	Passwords débiles	Secuestro de cuentas	MFA
A08	Software and Data Integrity Failures	Actualizaciones inseguras	Dependencias manipuladas	Malware	Firmas digitales
A09	Security Logging Failures	No registrar eventos	Ataques invisibles	Difícil detección	Logs y monitoreo
A10	SSRF	Servidor hace requests maliciosos	Acceso interno vía URL	Robo interno	Validar URLs

Casos reales

Caso 1 — Equifax (2017)

Empresa	Año	Como fue explotada
Equifax	2017	Los atacantes explotaron una vulnerabilidad conocida en el framework Apache Struts que no había sido actualizada por la empresa. Esto permitió el acceso no autorizado a servidores internos y el robo de información personal y financiera de aproximadamente 147 millones de personas, incluyendo nombres, números de seguro social y datos bancarios.

Impacto:

Este caso tuvo un impacto mundial debido a la enorme cantidad de información comprometida. La empresa enfrentó demandas millonarias, pérdida de confianza pública y uno de los mayores escándalos de seguridad de la década.

Caso 2 — SolarWinds (2020)

Empresa	Año	Como fue explotada
SolarWinds	2020	Los atacantes comprometieron el proceso de actualización del software Orion de SolarWinds e insertaron código malicioso dentro de una actualización oficial. Miles de organizaciones instalaron el software infectado creyendo que era legítimo, permitiendo acceso remoto y espionaje dentro de redes gubernamentales y corporativas.

Impacto:

Fue considerado uno de los ataques de cadena de suministro más graves de la historia. Afectó agencias gubernamentales de Estados Unidos y grandes empresas tecnológicas, demostrando el enorme riesgo de confiar en software de terceros sin mecanismos sólidos de verificación e integridad.

Código vulnerable vs seguro

1. SQL Injection

Vulnerable:

```
const query = "SELECT * FROM users WHERE user = '" + username + "'";
```

¿Por qué es vulnerable?

El código es vulnerable porque concatena directamente la información que escribe el usuario dentro de la consulta SQL. Esto permite que un atacante inserte instrucciones SQL maliciosas y modifique el comportamiento de la consulta original.

Por ejemplo, un atacante podría escribir código especial para acceder a toda la base de datos, eliminar información o incluso evitar el inicio de sesión. El problema principal es que la aplicación no valida ni separa correctamente los datos ingresados por el usuario de las instrucciones SQL reales.

Seguro:

```
const query = "SELECT * FROM users WHERE user = ?";  
db.query(query, [username]);
```

¿Por qué es seguro?

El código seguro utiliza consultas parametrizadas, lo que significa que los datos del usuario son tratados únicamente como información y no como instrucciones SQL. De esta manera, aunque el usuario escriba caracteres maliciosos, la base de datos no los ejecutará como comandos.

Esto evita que los atacantes alteren la consulta y protege la integridad de la base de datos.

2. Broken Access Control

Vulnerable:

```
app.get("/admin", (req,res)=>{  
  res.send("Panel admin");  
});
```

¿Por qué es vulnerable?

El código vulnerable permite acceder directamente a una ruta de administrador sin verificar si el usuario realmente tiene permisos para entrar. Esto significa que cualquier persona que conozca la URL podría acceder a información o funciones restringidas.

La vulnerabilidad ocurre porque no existe un mecanismo de autenticación o validación de roles antes de mostrar el contenido protegido.

Seguro:

```
app.get("/admin", authMiddleware, adminMiddleware);
```

¿Por qué es seguro?

El código seguro agrega middleware de autenticación y autorización antes de permitir el acceso a la ruta. Primero se verifica que el usuario haya iniciado sesión y después se valida que tenga permisos de administrador.

Esto garantiza que solamente usuarios autorizados puedan acceder a funciones sensibles del sistema.

3. Contraseñas inseguras

Vulnerable:

```
password = "123456";
```

¿Por qué es vulnerable?

El código vulnerable guarda las contraseñas en texto plano dentro de la base de datos. Si un atacante obtiene acceso a la información almacenada, podrá ver todas las contraseñas exactamente como fueron escritas por los usuarios.

Además, muchas personas reutilizan contraseñas en diferentes plataformas, por lo que una filtración podría comprometer otras cuentas personales.

Seguro:

```
bcrypt.hash(password, 10);
```

¿Por qué es seguro?

El código seguro utiliza hashing mediante algoritmos como bcrypt antes de almacenar las contraseñas. El hashing transforma la contraseña en una cadena cifrada irreversible.

De esta forma, aunque la base de datos sea comprometida, los atacantes no podrán conocer fácilmente las contraseñas originales de los usuarios.

Ranking personal

No. Ranking	Vulnerabilidad	Justificación
1	Injection	Considero que esta es una de las vulnerabilidades más peligrosas porque permite que un atacante manipule directamente consultas o comandos del sistema. Un ataque de SQL Injection puede provocar robo de información, modificación de bases de datos e incluso control total de la aplicación. Además, sigue siendo una vulnerabilidad muy común en aplicaciones mal desarrolladas.
2	Broken Access Control	Esta vulnerabilidad es crítica porque un usuario sin permisos puede acceder a funciones restringidas o información sensible simplemente modificando URLs o peticiones. Puede comprometer completamente la privacidad y seguridad de los usuarios, especialmente en sistemas bancarios, médicos o administrativos.
3	Vulnerable Components	Muchas empresas dependen de librerías y frameworks externos, por lo que usar componentes desactualizados representa un gran riesgo. Un atacante puede aprovechar vulnerabilidades conocidas públicamente para comprometer sistemas completos sin necesidad de técnicas avanzadas. Casos como Log4Shell demostraron el enorme impacto que puede tener este problema.
4	Cryptographic Failures	La mala protección de datos sensibles puede ocasionar filtraciones masivas de información personal, financiera o credenciales. Si las contraseñas no están cifradas correctamente o se usan protocolos inseguros, los atacantes pueden obtener información crítica fácilmente. Esta vulnerabilidad afecta directamente la confianza de los usuarios.
5	SSRF	El SSRF es muy peligroso porque permite que el servidor realice solicitudes internas manipuladas por el atacante. Esto puede exponer servicios privados, infraestructura interna o información confidencial que normalmente no sería accesible desde internet. Además, es una vulnerabilidad difícil de detectar si no se validan correctamente las URLs y peticiones externas.

Conclusión

Durante esta práctica comprendí la gran importancia que tiene el OWASP Top 10 dentro del desarrollo de aplicaciones web. Antes de realizar esta investigación, conocía algunos riesgos de seguridad, pero no imaginaba el impacto tan grande que pueden tener vulnerabilidades como SQL Injection, fallas de autenticación o errores en el control de acceso. Analizar casos reales me ayudó a entender que cualquier sistema puede ser atacado si no se implementan medidas de seguridad adecuadas.

También aprendí que muchas vulnerabilidades ocurren por errores simples en el desarrollo, malas configuraciones o falta de actualización de componentes. Por eso considero que la seguridad debe formar parte del proceso de desarrollo desde el inicio y no solamente agregarse al final del proyecto. Además, entender la diferencia entre código vulnerable y código seguro me permitió identificar mejores prácticas para proteger aplicaciones y datos de los usuarios.

Como estudiante y futuro desarrollador, considero que conocer el OWASP Top 10 es fundamental para crear sistemas más seguros, confiables y profesionales. Aplicar estos conocimientos me ayudará a reducir riesgos y desarrollar aplicaciones con una mejor calidad y mayor protección frente a posibles ataques informáticos.

Referencias

Iniseg. (2025, 13 junio). *Casos Reales de Ciberataques: Lecciones del Ataque a SolarWinds*.

Comunicación INISEG.

<https://www.iniseg.es/comunicacion-iniseg/blog/2025/06/02/casos-reales-de-ciberataques-lecciones-del-ataque-a-solarwinds/>

Academy, E. (s. f.). *Caso Equifax: uno de los mayores fallos de seguridad de la historia*.

<https://eud.academy/caso-equifax-uno-de-los-mayores-fallos-de-seguridad-de-la-historia/>

OWASP Top 10:2025. (s. f.). <https://owasp.org/Top10/2025/>

Team, O. A. S. P. (s. f.). *OWASP Top 10 API Security Risks – 2023 - OWASP API Security Top*

10. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>